МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
УНІВЕРСИТЕТ МИТНОЇ СПРАВИ ТА ФІНАНСІВ
ФАКУЛЬТЕТ ІННОВАЦІЙНИХ ТЕХНОЛОГІЙ
Кафедра комп’ютерних наук та інженерії програмного забезпечення

Практична робота №1

з дисципліни: «Тестування програмного забезапечення»

на тему: **«**Розроблення ПЗ з UI/UX та функціональними помилками»

Виконав:
Студент 4 курсу, групи ІПЗ22-1
Циганок М.В

Перевірив:
Кандидат технічних наук, доцент кафедри
Жульковська І.І

Дніпро
2026

1. Розроблення ПЗ з UI/UX та функціональними помилками

1.1. Аналіз практичної роботи

Мета роботи: «набути знання та навички з аналізу інтерфейсів і функціоналу програмних продуктів шляхом створення застосунку з допущеними помилками в дизайні та роботі; навчитися розуміти вплив недоліків на зручність використання, ефективність і задоволеність користувачів»

Тематикою програмного забезпечення було обрано – список завдань. Для реалізації ПЗ було використано мову програмування Python та бібліотеку «customtkinter»

Розороблений ПЗ має наступний функціонал: додавання нових задач, відображення списку задач, видалення задач, позначення як виконаних, збереження задач у файл

1.2. Функціональні помилки

Помилка 1. Додавання пустої задачі

Програма дозволяє додавати задачі без введеного тексту, тобто користувач може натиснути кнопку додавання, навіть якщо поле вводу порожнє. Це відбувається через те, що в коді відсутня перевірка введеного значення перед додаванням задачі до списку. У результаті в списку з’являються порожні елементи, що погіршує зручність використання програми та може заплутати користувача. Для виправлення цієї помилки необхідно додати перевірку на порожній рядок, наприклад, перевіряти чи введений текст

не є пустим або не складається лише з пробілів перед тим, як додавати його до списку задач.

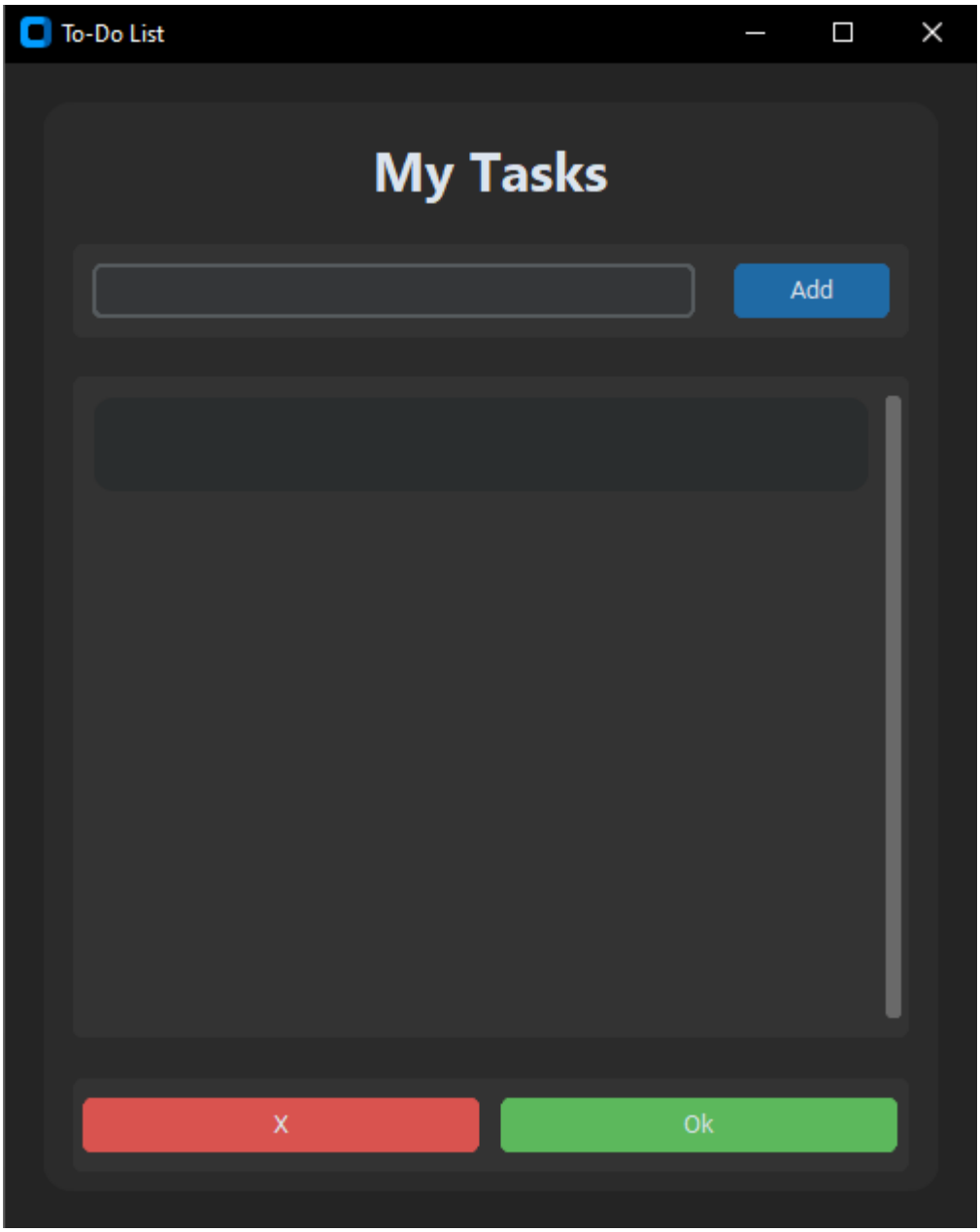

Рисунок 1.1 – Демонстрація додавання «пустої» задачі

Помилка 2. Видалення неправильної задачі

При натисканні кнопки видалення програма видаляє не ту задачу, яку обрав користувач, а останній елемент у списку. Така поведінка виникає через те, що використовується метод видалення без зазначення індексу, тобто pop() видаляє останній елемент незалежно від вибору. У результаті порушується логіка роботи програми, що може викликати плутанину та незадоволення користувача. Для виправлення цієї помилки необхідно видаляти задачу саме за індексом вибраного елемента, щоб дія відповідала очікуванням користувача.

Додамо 2 задачі: «Зробити практичну роботу з Тестування програмного забезапечення», «Оформити звіт з практичної роботи»

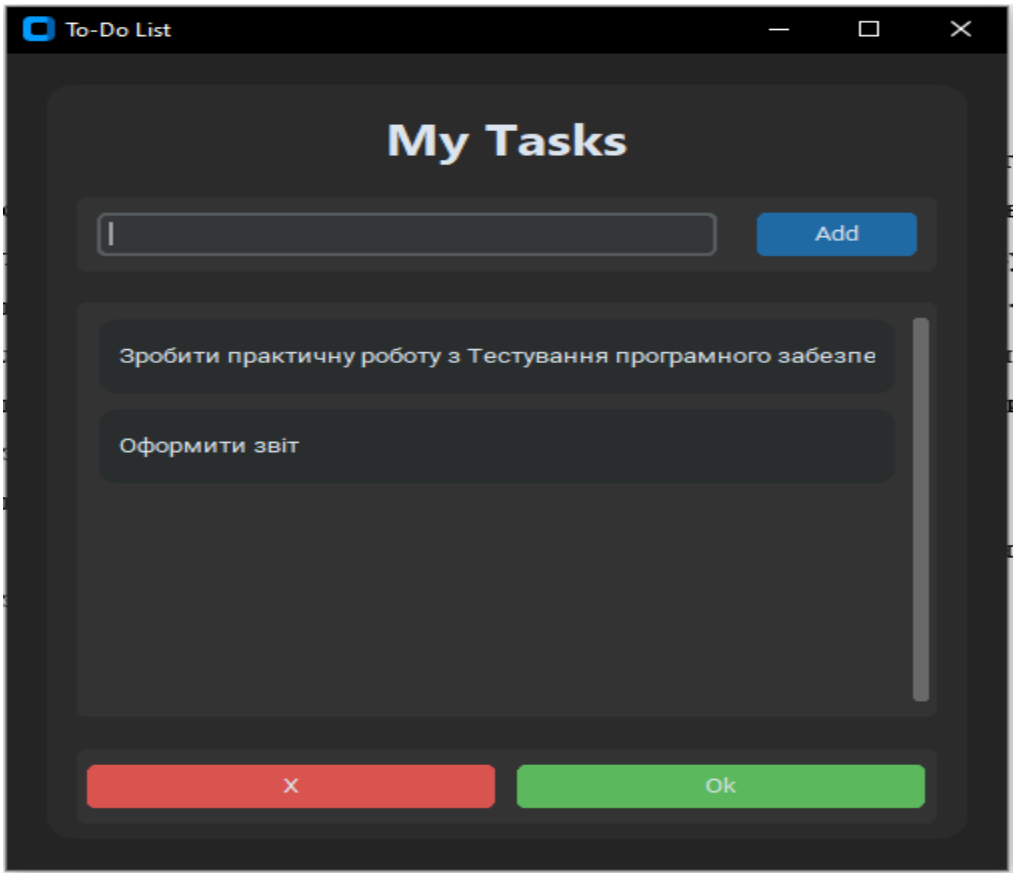

Рисунок 1.2 – Додавання завдань

Спробуємо видалити 1-ше завдання, та побачимо, що функціонал виделання працює некоректно

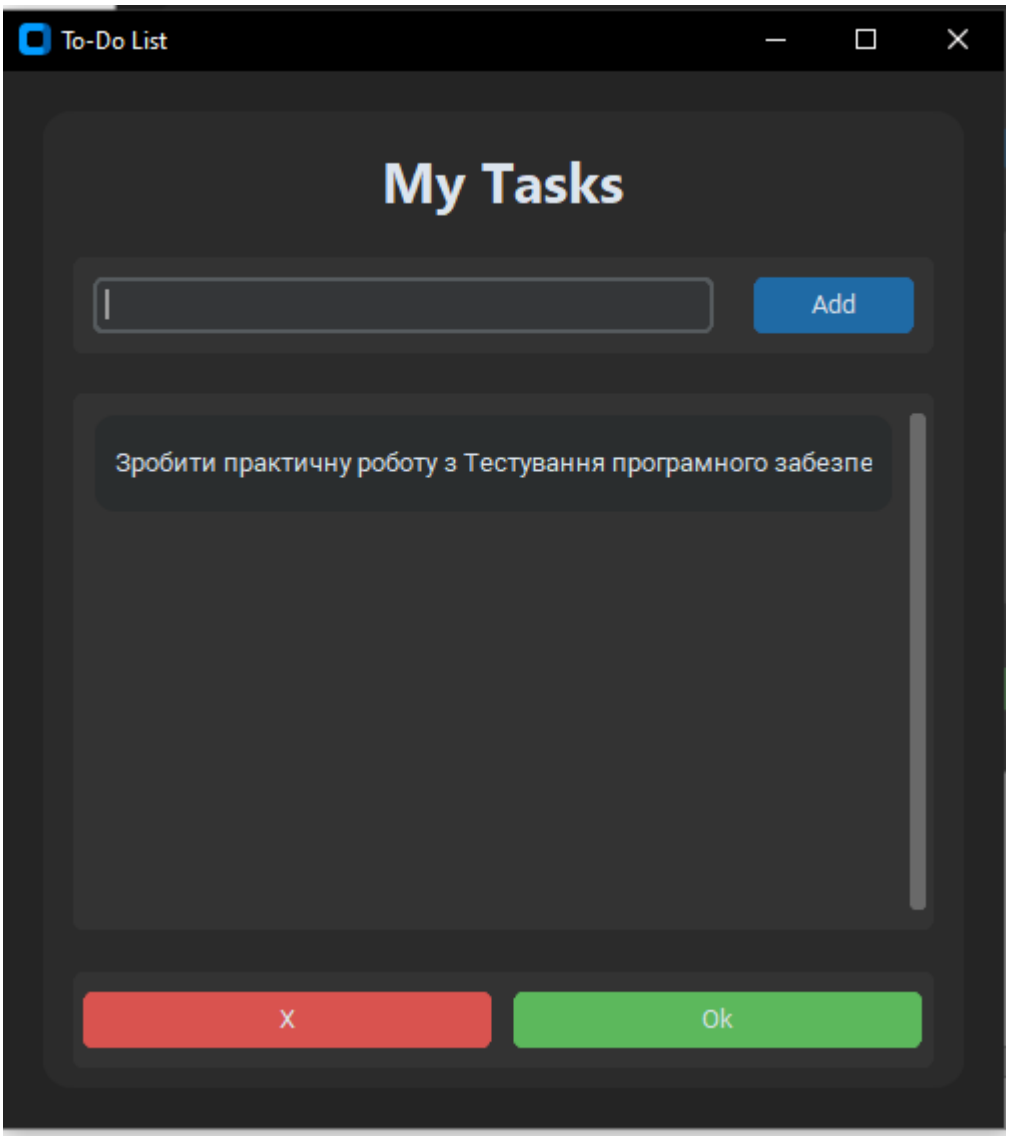

Рисунок 1.3 – Видалення завдання

1.3. UI/UX помилки

Помилка 1. Відсутність підтвердження видалення

При видаленні задачі не відображається вікно підтвердження, тому дія виконується одразу після натискання кнопки. Через це користувач може випадково видалити важливу інформацію без можливості швидко скасувати дію. Відсутність підтвердження знижує безпеку та зручність використання програми, оскільки не дає користувачу додаткового контролю над критичними діями. Для виправлення цієї проблеми необхідно додати діалогове вікно підтвердження перед видаленням задачі, щоб користувач міг підтвердити або скасувати виконання операції.

Помилка 2. Відсутність індикації вибраної задачі

У списку задач не відображається, яка саме задача вибрана, тому користувач не може зрозуміти, до якого елемента буде застосована певна дія. Через це підвищується ризик помилок при роботі з програмою та знижується її зручність. Для виправлення цієї проблеми необхідно додати візуальне виділення вибраного елемента, щоб користувач одразу бачив, яка задача активна.

Помилка 3. Незрозумілі підписи кнопок

Кнопки мають неінформативні назви, “X” та “OK”, через що користувачу складно зрозуміти їх призначення. Такий підхід знижує зручність використання програми та може призводити до помилкових дій, оскільки інтерфейс стає менш зрозумілим. Для виправлення цієї проблеми необхідно використовувати чіткі та описові назви кнопок, такі як Delete або Mark Done, щоб їх функціональність була очевидною для користувача.

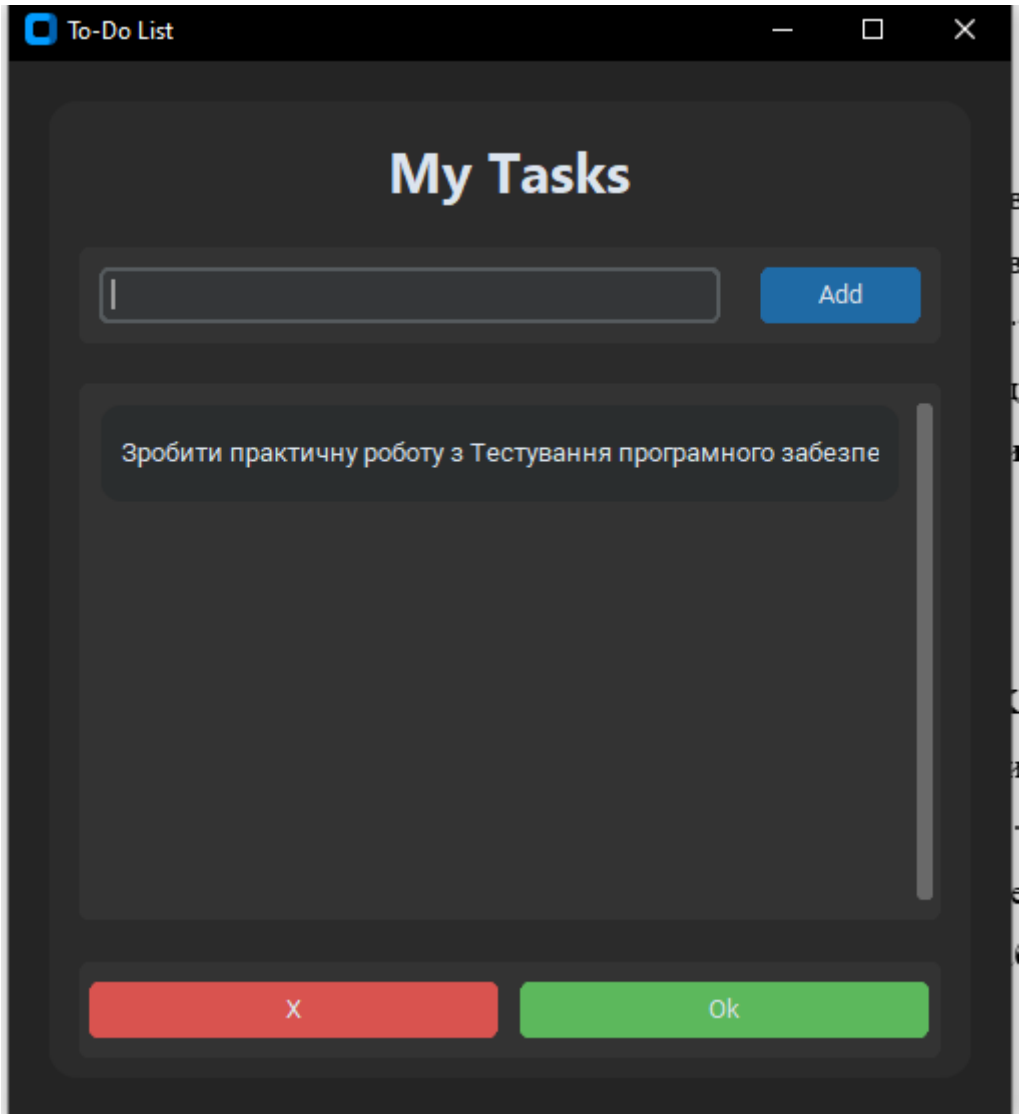

Рисунок 1.4 – Головне меню

1.4. Контрольні запитання

1. Що таке UI та UX, у чому їхня відмінність та які основні принципи професійного UI/UX дизайну?

UI (User Interface) — це користувацький інтерфейс, тобто зовнішній вигляд програми: кнопки, поля вводу, кольори, розташування елементів. UX (User Experience) — це користувацький досвід, тобто те, наскільки зручно, логічно та ефективно користувачу працювати з програмою. UI відповідає за

візуальну частину, а UX — за зручність і логіку використання. Основними принципами професійного UI/UX є простота, зрозумілість, логічність, консистентність, доступність та надання зворотного зв’язку користувачу.

2. Які типові помилки можна допустити при розташуванні елементів інтерфейсу?

Типовими помилками є хаотичне розміщення елементів, відсутність логічної структури, занадто щільне розташування компонентів, неправильна ієрархія, а також нерівні відступи. Це ускладнює сприйняття інтерфейсу та знижує зручність користування.

3. Чому та як надмірне використання кольорів та анімацій може погіршити UX?

Надмірна кількість кольорів та анімацій може відволікати користувача, створювати візуальний шум і ускладнювати сприйняття інформації. Занадто яскраві або несумісні кольори можуть знижувати читабельність тексту. Надмірні анімації можуть уповільнювати роботу програми та викликати дискомфорт. Тому дизайн повинен бути мінімалістичним і збалансованим.

4. Що таке консистентність інтерфейсу і чому вона важлива?

Консистентність — це однаковість стилю, поведінки та логіки елементів інтерфейсу у всій програмі. Наприклад, однакові кнопки повинні виглядати та працювати однаково. Консистентність важлива, оскільки вона робить програму передбачуваною, зручною та зрозумілою для користувача.

5. Що таке функціональні та нефункціональні вимоги до програмного забезпечення і в чому їх різниця?

Функціональні вимоги описують, що саме повинна робити програма. Нефункціональні вимоги описують, як саме система повинна працювати. Різниця полягає в тому, що функціональні вимоги визначають поведінку системи, а нефункціональні — її якість та характеристики.

6. Як впливає відсутність зворотного зв’язку після виконання функції?

Відсутність зворотного зв’язку може викликати невпевненість користувача, оскільки він не розуміє, чи була дія виконана успішно. Це знижує довіру до програми та погіршує користувацький досвід. Зворотний зв’язок може бути у вигляді повідомлень, змін у інтерфейсі або візуальних індикаторів.

7. Що означає неповне виконання вимог до функції?

Неповне виконання вимог означає, що функція реалізована частково або працює не так, як зазначено у технічному завданні. Це може призводити до помилок у роботі програми або до неможливості використання певних можливостей.

8. Як можна перевірити відповідність реалізованої функціональності технічному завданню?

Відповідність можна перевірити шляхом тестування програми, порівняння її роботи з вимогами технічного завдання, проведення функціональних тестів та перевірки всіх сценаріїв використання. Також можна використовувати ручне тестування та аналіз результатів роботи програми.

9. Чому важлива наявність повідомлень про помилки та їх коректність?

Повідомлення про помилки допомагають користувачу зрозуміти причину проблеми та способи її виправлення. Коректні та зрозумілі повідомлення підвищують зручність використання програми та зменшують кількість неправильних дій. Вони є важливою частиною якісного UX та професійного програмного забезпечення.